Chapter 3

Object Tracking Engine

Gautami Yeshwant Deshpande (251123)
Abhipsa Mishra (251101)

3.1 Introduction and Problem Statement

Detecting and following moving objects in video is a basic building block of video surveillance, traffic monitoring, sports analytics and robotics. A person watching a road camera can easily tell that "the white car in the middle lane is moving forward quickly"; a program must reproduce that judgement from a grid of pixels, frame after frame.

The **Object Tracking Engine** is a C++17 desktop application that reads a video file, finds the moving objects in every frame, assigns each of them a persistent identity number, and reports how each one is moving: its speed, its direction of travel and whether it is moving or stationary. The results are shown on a live, annotated video window and also written to the console as a text log.

3.1.1 Problem Statement

Given a video stream, the system must:

1. **detect** the regions of every frame that contain motion;
2. **associate** detections in consecutive frames so that the same physical object keeps the same ID while it stays in view;
3. **analyse** the motion of each tracked object (speed, direction, status); and
4. **present** the result to the user visually and as text.

The difficulty lies in the imperfect nature of real video: compression noise creates false detections, objects enter and leave the scene, and an object may be missed for a few frames. Besides a working result, the project aims at a clean object-oriented design in which every responsibility (reading frames, detecting, tracking, analysing, drawing) lives in its own class with a small, well-defined interface.

3.1.2 What the System Does (End-User View)

The user starts the program, optionally passing the path of a video file; if no path is given, `Video.mp4` in the working directory is used. A window titled "Object Tracking Engine" opens and plays the video. For every moving object the user sees:

- a green rectangle around the object and a red dot at its centroid;

- a text label of the form `ID: <n> <STATUS> <DIRECTION> <speed> px/s`; and
- a blue line showing the path the object has followed so far.

At the same time, one log line per object per frame is printed to the console. Pressing `q` stops the program.

3.1.3 Software Requirements

The development and runtime environment of the project is summarised in Table 3.1.

Item	Details
Language	C++17 (`CMAKE_CXX_STANDARD 17`, extensions off)
Library	OpenCV 5 (components: `core`, `geometry`, `imgproc`, `video`, `videoio`, `highgui`)
Build system	CMake 3.16 or newer, with a `CMakePresets.json` preset for Visual Studio
Compiler	MSVC (enforced in `CMakeLists.txt` so that it matches the installed OpenCV binaries)
Platform	Windows, x64
Input	Any video format readable by OpenCV `videoio`; the demonstration uses a traffic-camera clip, `Video.mp4`

Table 3.1: Development and runtime environment

3.1.4 Technology Stack

Table 3.2 lists the technologies used to build and run the project, the role each one plays, and the reason it was chosen.

Layer	Technology	Role and reason
Language	C++17	Compiled and fast enough for about 30 frames per second; classes, operator overloading, `const`-correctness, RAII.
Standard library	`vector`, `map`, `string`, `filesystem`, `cmath`	Detections, tracked objects and trajectories; labels; video path; distance and speed.
Vision library	OpenCV 5 (`core`, `geometry`, `imgproc`, `video`, `videoio`, `highgui`)	Video reading, `cv::Mat`, MOG2 background subtraction, morphology, contours, drawing and display window.
Build system	CMake 3.16+ with `CMakePresets.json`	Finds OpenCV and links the eight source files into one executable.
Compiler / IDE	MSVC (Visual Studio)	Matches the installed OpenCV binaries; enforced in `CMakeLists.txt`.
Platform	Windows, x64	Target for the installed OpenCV libraries and DLLs.
Input / output	`Video.mp4`; OpenCV window and console	Any format readable by OpenCV `videoio`; results shown live and logged (no video file saved).

Table 3.2: Technology stack of the Object Tracking Engine

3.2 Scope and Feature Set

- **Moving-object detection:** Each frame is passed through an adaptive Gaussian-mixture background subtractor (MOG2). The foreground mask is cleaned with morphological opening and closing, and every sufficiently large connected region is reported as a `BoundingBox`.
- **Noise and size filtering:** Shadow pixels and weak foreground responses are removed by thresholding, small speckles by morphology, and regions smaller than a configurable minimum area (500 pixels by default) are discarded.
- **Persistent ID tracking:** Detections are matched to existing tracked objects by the distance between centroids (at most 80 pixels by default). Unmatched detections create new objects with fresh, increasing IDs.
- **Tolerance to missed detections:** An object that is not matched keeps its identity for up to five consecutive frames before it is deleted.
- **Motion analysis:** Speed (pixels per second, computed from real video timestamps), direction (`LEFT`, `RIGHT`, `UP`, `DOWN`, `STATIONARY`) and status (`MOVING` or `STATIONARY`) are computed for each object in every frame.
- **Visualisation:** Bounding box, centroid, text label and full trajectory are drawn on the frame. The display is scaled down automatically to fit within 1280×720 pixels.
- **Console logging:** The frame number, ID, position, speed, direction, status and missed-frame count of every object are printed for later inspection.
- **Configurable components:** The minimum area, matching distance, missed-frame limit and movement threshold are constructor parameters with sensible defaults.
- **Modular object-oriented design:** Seven cohesive classes with clear responsibilities, communicating through small, `const`-correct interfaces.

Out of scope. The system does not convert pixel speeds to real-world units (no camera calibration), does not classify objects (car, bus, person), does not re-identify an object after a long disappearance, and does not save the annotated video to disk.

3.3 Object-Oriented Architecture

The design follows a few principles that recur throughout the code: *encapsulation* (data members are private and exposed through `const` accessors), *single responsibility* (each class does one job), *composition* (larger objects own the smaller ones they are built from) and *loose coupling* (classes exchange small value objects such as `Frame` and `BoundingBox` instead of sharing internal state).

3.3.1 Overall Pipeline

Figure 3.1 shows how data flows through the system for every frame. The `main()` function acts only as the coordinator: it owns one instance of each processing class and passes the output of one stage to the next.

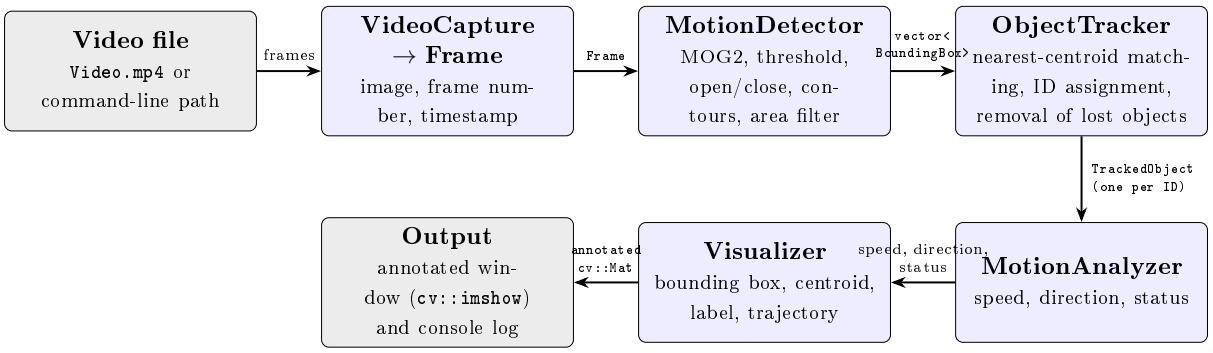


Figure 3.1: Processing pipeline of the Object Tracking Engine (one iteration per video frame; the second row is read right to left).

3.3.2 Class Design and Associations

The program consists of seven classes. Table 3.3 summarises their responsibilities and Figure 3.2 shows the relations between them. The OOP concepts used by these classes are demonstrated with code in Section 3.4.

Class	Responsibility	Main collaborators
`Frame`	Immutable value object holding one video frame: a private copy of the image, its frame number and its timestamp in seconds.	`MotionDetector`, `main()`
`BoundingBox`	Axis-aligned rectangle (x, y, width, height) with centroid and area queries; supports `+` and `-` with a `cv::Point2f` offset.	`MotionDetector`, `TrackedObject`, `ObjectTracker`
`MotionDetector`	Finds moving regions in a frame using MOG2 background subtraction and contour analysis.	`Frame`, `BoundingBox`
`TrackedObject`	State of one tracked object: ID, current box, previous and current centroid, trajectory, timestamps, speed, direction, status and missed-frame counter.	`BoundingBox`
`ObjectTracker`	Maintains the collection of tracked objects; matches detections to objects, creates new objects and removes lost ones.	`TrackedObject`, `BoundingBox`
`MotionAnalyzer`	Computes speed, direction and status of a tracked object from its last two positions and timestamps.	`TrackedObject`
`Visualizer`	Draws the box, centroid, label and trajectory of an object onto a frame.	`TrackedObject`

Table 3.3: Classes and their responsibilities

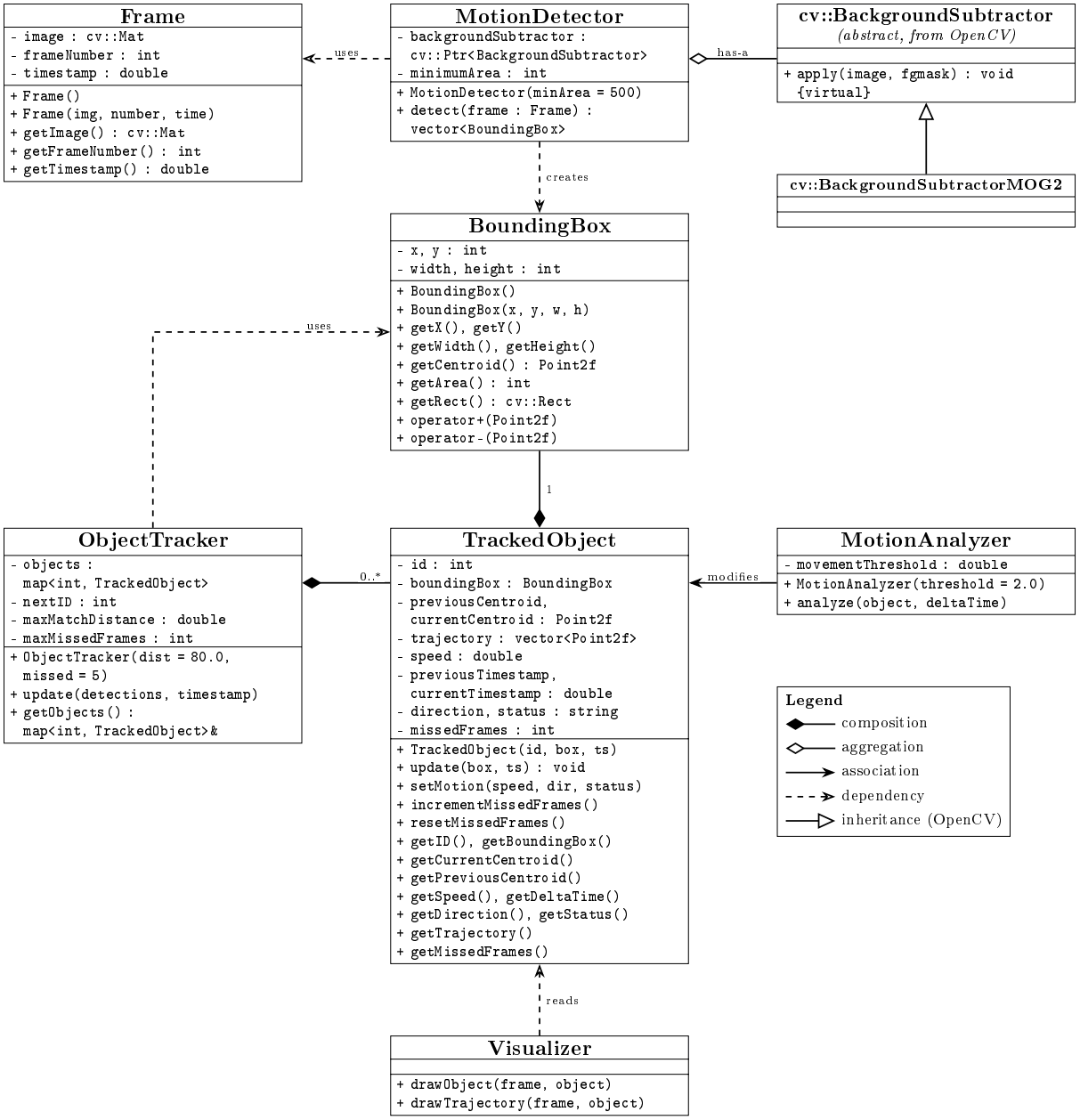


Figure 3.2: UML class diagram. The seven project classes are linked by composition, association and dependency; the only inheritance involved belongs to OpenCV's background-subtractor interface.

Composition. `TrackedObject` *owns* its `BoundingBox` and its trajectory (a `std::vector<cv::Point2f>`) as ordinary data members; they cannot exist without the object. `ObjectTracker` owns every `TrackedObject` by value inside a `std::map<int, TrackedObject>`: when the tracker erases an entry, the object and everything it contains are destroyed with it. Likewise `Frame` owns a private deep copy of its image.

Aggregation. `MotionDetector` holds its background model through a `cv::Ptr<cv::BackgroundSubtractor>`, a reference-counted smart pointer. The detector uses the subtractor as a part of itself, but the part is managed by shared ownership rather than embedded by value, which is why it is drawn as aggregation.

Association and dependency. `MotionAnalyzer` receives a `TrackedObject` by non-`const` reference and updates it through `setMotion()`. `Visualizer` only reads a `TrackedObject`. `MotionDetector` depends on `Frame` (its input) and creates `BoundingBox` values (its output), and `ObjectTracker` depends on `BoundingBox` for the detections it receives. These are short-lived "uses" relations; none of the classes stores a long-term pointer to another.

Inheritance. The project does not define its own class hierarchy, because the problem is naturally a chain of independent processing steps rather than a family of related types. Inheritance and run-time polymorphism enter only through OpenCV: `MotionDetector` calls `apply()` through a pointer to the abstract base class `cv::BackgroundSubtractor`, while the object behind it is a `BackgroundSubtractorMOG2` (Section 3.4.8).

3.4 OOP Concepts Demonstrated in the Code

This section shows, using the actual source code, how the main object-oriented concepts are applied. Table 3.4 at the end of the section summarises where each concept appears.

3.4.1 Encapsulation and Data Hiding

Every class keeps its data private and offers read access through `const` member functions. `BoundingBox` (Listing 3.1) is a typical example: no outside code can put it in an inconsistent state, and the derived values (centroid, area, `cv::Rect`) are computed on demand rather than stored.

```
class BoundingBox
{
private:
    int x;
    int y;
    int width;
    int height;

public:
    BoundingBox();
    BoundingBox(int x, int y, int width, int height);

    int getX() const;
    int getY() const;
    int getWidth() const;
    int getHeight() const;

    cv::Point2f getCentroid() const;
    int getArea() const;
    cv::Rect getRect() const;

    // Operator overloading
    BoundingBox operator+(const cv::Point2f& offset) const;
    BoundingBox operator-(const cv::Point2f& offset) const;
};
```

Listing 3.1: `BoundingBox.h`: private data, public `const` interface.

`TrackedObject` goes one step further: its state can only change through the controlled operations `update()`, `setMotion()`, `incrementMissedFrames()` and `resetMissedFrames()`. In particular, `update()` guarantees that the previous and current centroids and timestamps always move forward together, which the analyser relies on (Listing 3.2).

```
void TrackedObject::update(const BoundingBox& newBox, double
    timestamp)
{
    previousCentroid = currentCentroid;
    previousTimestamp = currentTimestamp;
    currentTimestamp = timestamp;

    boundingBox = newBox;
    currentCentroid = newBox.getCentroid();
    trajectory.push_back(currentCentroid);

    missedFrames = 0;
}
```

Listing 3.2: `TrackedObject::update()` keeps position, time and history consistent.

3.4.2 Constructors and Constructor Overloading

`BoundingBox` provides a default constructor (an empty box at the origin) and a parameterised one. `Frame` does the same (Listing 3.3). In `Frame`, the parameterised constructor calls `clone()` on the image, so the `Frame` owns an independent copy: later drawing on the original image in `main()` cannot affect the data that the detector processes.

```
BoundingBox::BoundingBox()
{
    x = 0;
    y = 0;
    width = 0;
    height = 0;
}

BoundingBox::BoundingBox(int x, int y, int width, int height)
{
    this->x = x;
    this->y = y;
    this->width = width;
    this->height = height;
}

Frame::Frame(const cv::Mat& img, int number, double time)
{
    image = img.clone();
    frameNumber = number;
    timestamp = time;
}
```

Listing 3.3: Overloaded constructors of `BoundingBox` and `Frame`.

3.4.3 Default Arguments

The tunable classes take their parameters through constructors with default values, so a component can be created with a recommended configuration or customised at the call site (Listing 3.4). In `main()` the defaults are overridden explicitly: `MotionDetector detector(500);`, `ObjectTracker tracker(80.0, 5);` and `MotionAnalyzer analyzer(2.0);`.

```
MotionDetector(int minArea = 500);                  // MotionDetector.h
MotionAnalyzer(double threshold = 2.0);             // MotionAnalyzer.h
ObjectTracker(double maxDistance = 80.0,            // Tracker.h
              int maxMissed = 5);
TrackedObject(int id, const BoundingBox& box,       // TrackedObject.h
              double timestamp = 0.0);
```

Listing 3.4: Constructors with default arguments (declarations).

3.4.4 Operator Overloading

`BoundingBox` overloads `+` and `-` so that a box can be translated by a `cv::Point2f` offset with natural syntax: `BoundingBox moved = box + offset;`. Both operators are `const` and return a new box, leaving the original unchanged; the size is preserved. The floating-point offset is truncated to whole pixels because the box stores integer coordinates (Listing 3.5).

```
cv::Point2f BoundingBox::getCentroid() const
{
    return cv::Point2f(
        x + width / 2.0f,
        y + height / 2.0f
    );
}

BoundingBox BoundingBox::operator+(const cv::Point2f& offset) const
{
    return BoundingBox(
        x + static_cast<int>(offset.x),
        y + static_cast<int>(offset.y),
        width,
        height
    );
}

BoundingBox BoundingBox::operator-(const cv::Point2f& offset) const
{
    return BoundingBox(
        x - static_cast<int>(offset.x),
        y - static_cast<int>(offset.y),
        width,
        height
    );
}
```

Listing 3.5: Overloaded `+` and `-` operators, and the centroid computation.

3.4.5 Composition and Ownership of Objects

The tracker is the clearest example of composition. It owns all tracked objects in a `std::map` keyed by ID; `nextID` guarantees that IDs are never reused during a run. Because the objects are stored by value, creation and destruction of a `TrackedObject` are tied to the tracker's own bookkeeping, with no manual `new` or `delete` anywhere in the project (Listing 3.6).

```
class ObjectTracker
{
private:
    std::map<int, TrackedObject> objects;

    int nextID;

    double maxMatchDistance;
    int maxMissedFrames;

public:
    ObjectTracker(double maxDistance = 80.0, int maxMissed = 5);

    void update(const std::vector<BoundingBox>& detections,
                double timestamp = 0.0);

    std::map<int, TrackedObject>& getObjects();
};
```

Listing 3.6: `Tracker.h`: `ObjectTracker` owns its `TrackedObjects`.

3.4.6 Abstraction and Separation of Responsibilities

Each stage of the pipeline is hidden behind a one- or two-function interface. The caller does not need to know that detection uses MOG2 or that matching is greedy; it only needs `detect()`, `update()`, `analyze()` and `draw*()`. This makes it possible to replace the internals of one class (for example, a better matching algorithm in `ObjectTracker`) without touching the others. The main loop (Listing 3.7) reads almost like the problem statement.

```
Frame frame(image, frameNumber, timestamp);
// 1. Detect moving objects
std::vector<BoundingBox> detections = detector.detect(frame);
// 2. Track objects
tracker.update(detections, timestamp);
// 3. Analyze and visualize
for (auto& pair : tracker.getObjects()) {
    TrackedObject& object = pair.second;
    analyzer.analyze(object, object.getDeltaTime());
    visualizer.drawObject(image, object);
    visualizer.drawTrajectory(image, object);
    // ... console log ...
}
```

Listing 3.7: Core of the main loop: each stage is a single call on a dedicated object.

3.4.7 Const-Correctness

Accessors are declared `const`, inputs are passed as `const` references (`const BoundingBox&`, `const Frame&`, `const TrackedObject&`), and the trajectory is exposed as a `const std::vector<cv::Point2f>&` to avoid copying a growing vector. For instance, `MotionDetector::detect(const Frame&)` promises not to modify the frame, and `Visualizer::drawObject()` takes the object as `const`, documenting that drawing never changes tracking state. Only `MotionAnalyzer::analyze()` receives a modifiable `TrackedObject&`, which makes it easy to see which class is allowed to change motion data.

3.4.8 Polymorphism through an Abstract Interface and RAII

`MotionDetector` stores its background model as a `cv::Ptr<cv::BackgroundSubtractor>` (the abstract base class) and initialises it with the MOG2 implementation. The call to `apply()` is dispatched at run time to `BackgroundSubtractorMOG2`, so the detector would work unchanged with another subtractor such as KNN (Listing 3.8). The smart pointer also illustrates RAII: the model is released automatically when the detector is destroyed, just as `cv::Mat` images and the `cv::VideoCapture` release their resources without explicit clean-up code.

```
// MotionDetector.h
cv::Ptr<cv::BackgroundSubtractor> backgroundSubtractor;

// MotionDetector.cpp
MotionDetector::MotionDetector(int minArea)
{
    minimumArea = minArea;
    backgroundSubtractor = cv::createBackgroundSubtractorMOG2();
}
...
backgroundSubtractor->apply(image, foregroundMask);   // virtual call
```

Listing 3.8: Programming to the abstract `cv::BackgroundSubtractor` interface.

Concept	Location in the code
Encapsulation	Private members and `const` getters in `BoundingBox`, `Frame`, `TrackedObject`
Constructors	`BoundingBox()` / `BoundingBox(x,y,w,h)`; `Frame()` / `Frame(img,n,t)`
Default arguments	`MotionDetector`, `MotionAnalyzer`, `ObjectTracker`, `TrackedObject`
Operator overloading	`BoundingBox::operator+` and `operator-`
Composition	`ObjectTracker` → `TrackedObject` → `BoundingBox`; `Frame` → image copy
Aggregation	`MotionDetector` → `cv::Ptr<BackgroundSubtractor>` (shared ownership)
Abstraction	Seven classes with narrow public interfaces, coordinated by `main()`
Const-correctness	`const` member functions and `const` reference parameters throughout
Polymorphism	`BackgroundSubtractor::apply()` called through a base-class smart pointer (OpenCV)
RAII	`cv::Ptr`, `cv::Mat`, `std::map`, `std::vector`; no raw `new`/`delete`

Table 3.4: Where each OOP concept appears in the project

3.5 UML Behavioural Models

3.5.1 Use Case Diagram

Figure 3.3 shows the interactions between the user, the video source and the system. The *User* starts the tracker, views the annotated video, reads the console log and quits. The *Video file* is a secondary actor that supplies frames. Internally, annotating the video requires motion analysis, which requires tracking, which requires detection, which requires reading frames; this chain is expressed with *include* relations.

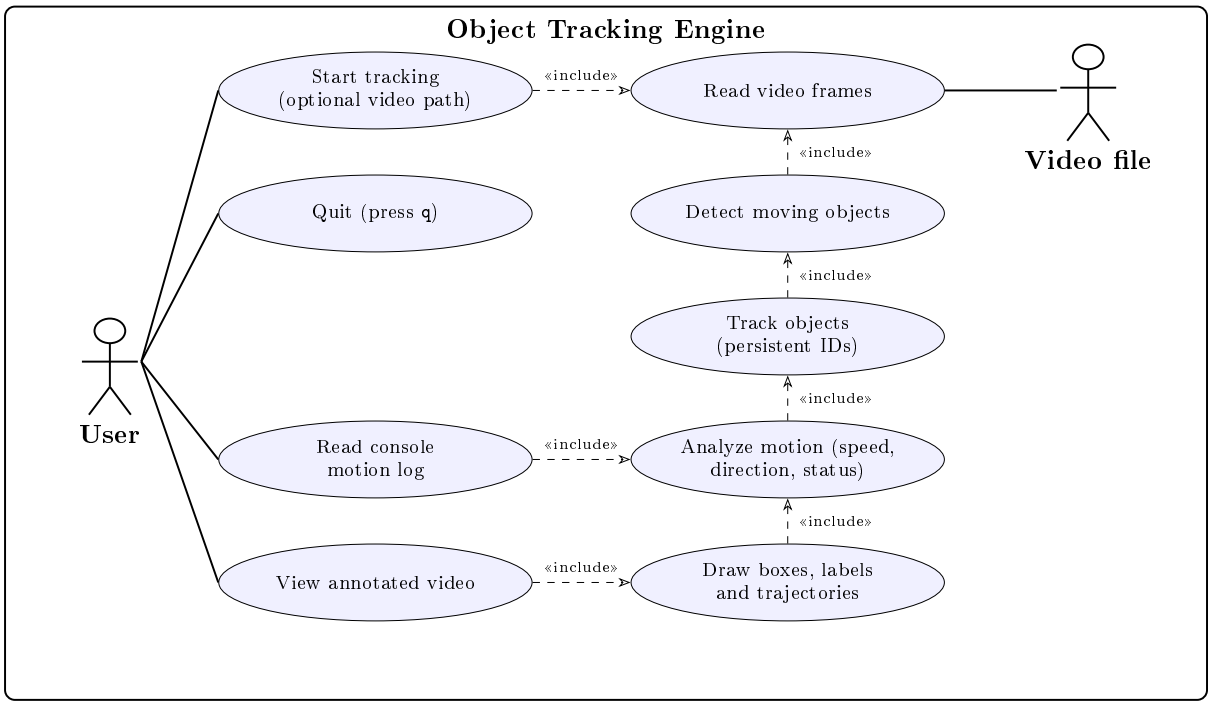


Figure 3.3: Use case diagram of the Object Tracking Engine.

3.5.2 Sequence Diagram

Figure 3.4 shows the messages exchanged while one frame is processed. `main()` reads the frame and wraps it in a `Frame`, asks the detector for bounding boxes, hands them to the tracker (which updates, creates or ages `TrackedObjects`), and finally, for every tracked object, calls the analyser and the visualiser, both of which talk to the `TrackedObject`.

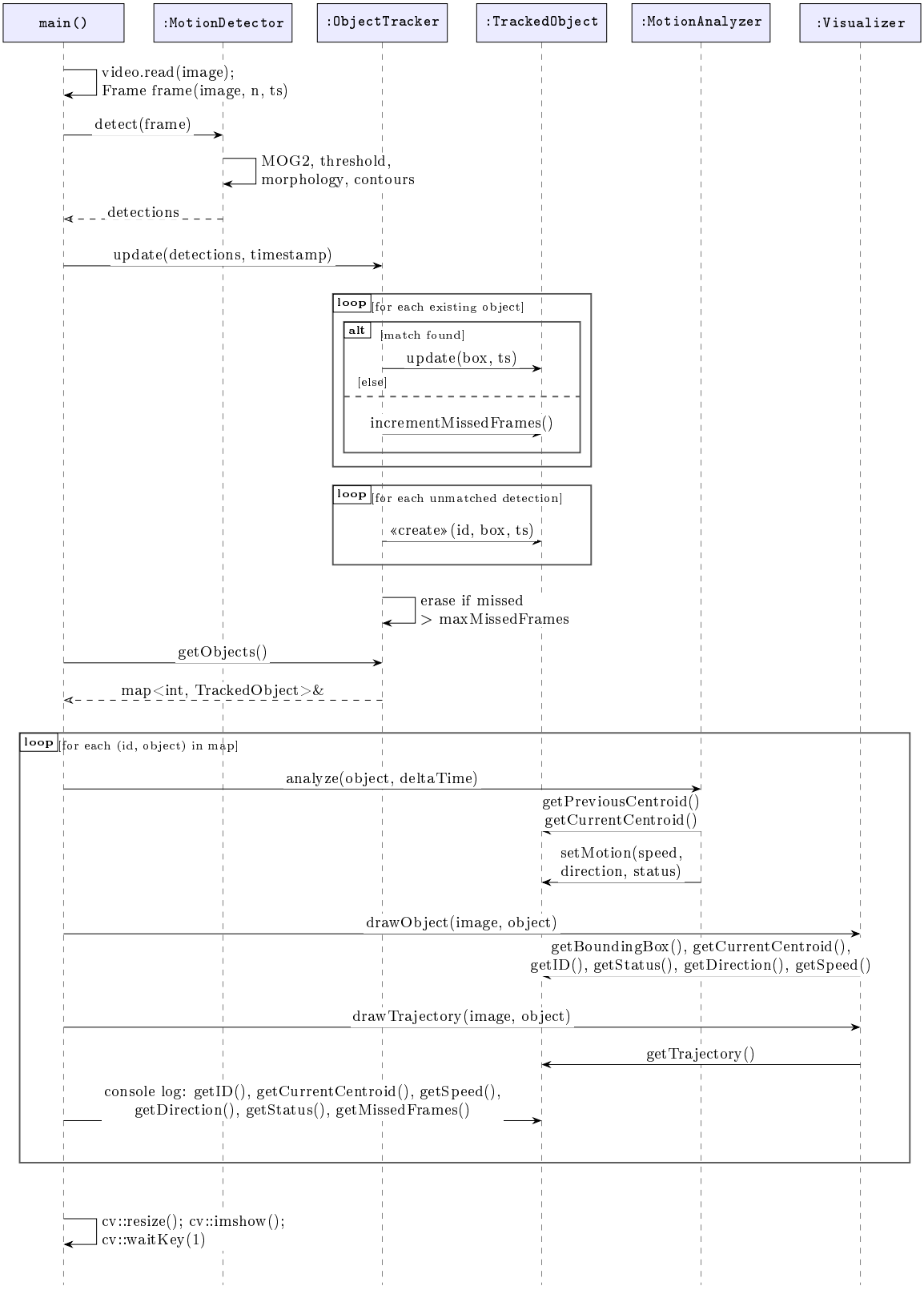


Figure 3.4: Sequence diagram for processing one video frame (simplified: for one tracked object).

3.5.3 Activity Diagrams

Figure 3.5 describes the control flow of the application as a whole, and Figure 3.6 zooms into the matching logic inside `ObjectTracker::update()`.

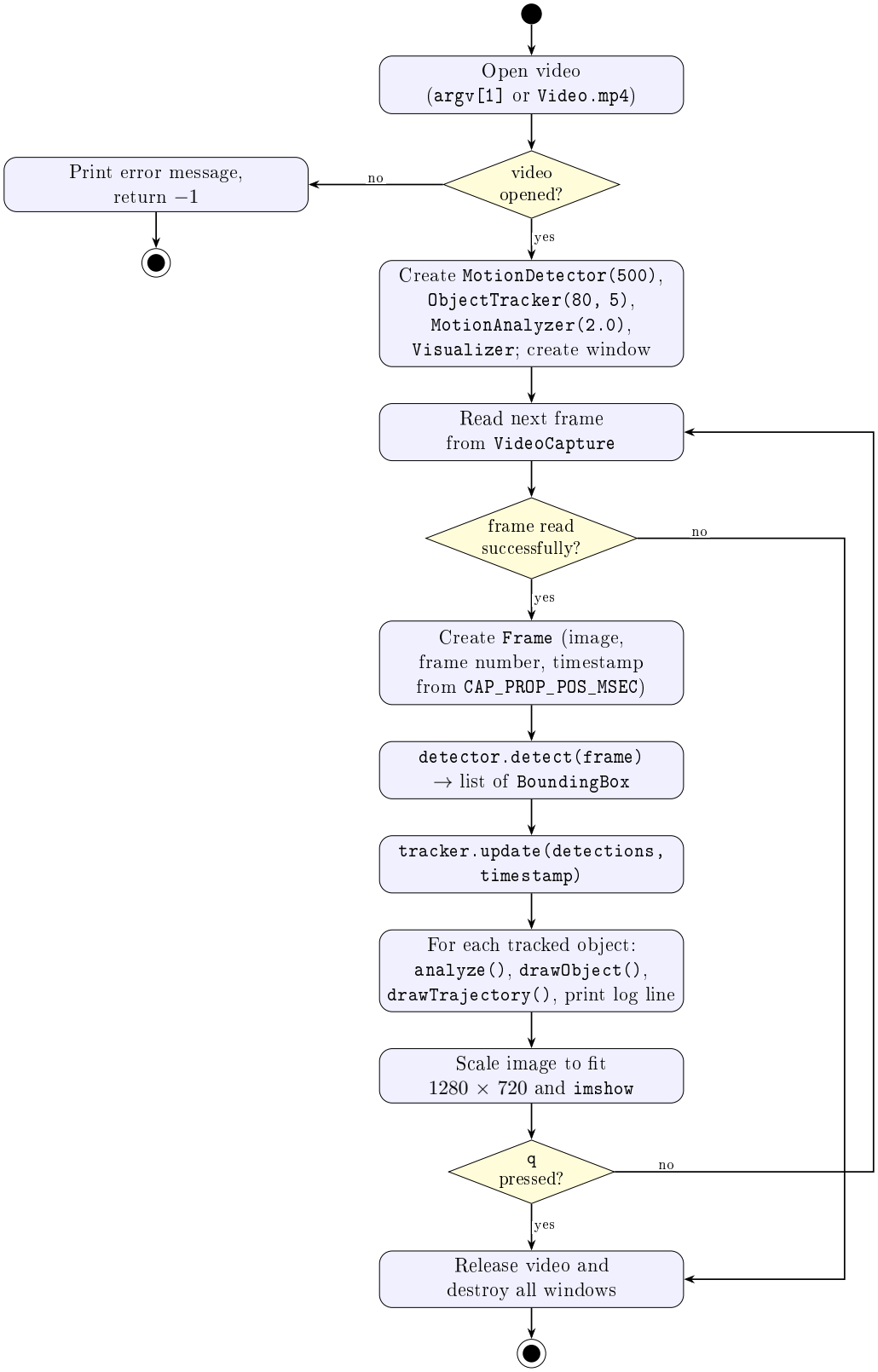


Figure 3.5: Activity diagram of the main processing loop.

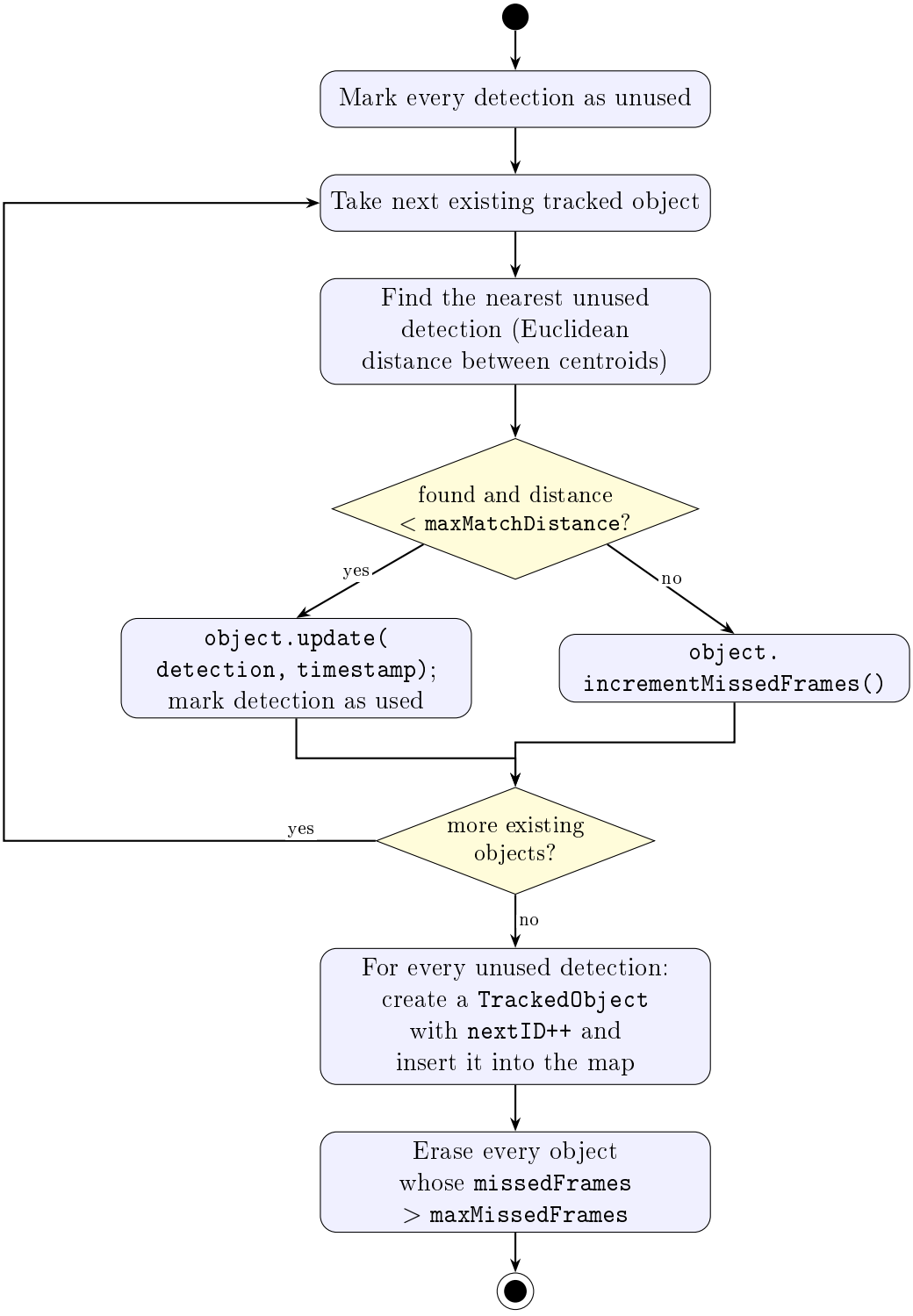


Figure 3.6: Activity diagram of `ObjectTracker::update()`.

3.6 Implementation Details

3.6.1 Motion Detection

`MotionDetector::detect()` turns a `Frame` into a list of bounding boxes in five steps (Listing 3.9).

1. **Background subtraction.** A MOG2 model (`cv::createBackgroundSubtractorMOG2()`, default parameters) is updated with the frame. MOG2 models each pixel as a mixture of Gaussians and marks it as background (0), shadow (127) or foreground (255).

2. **Thresholding.** `cv::threshold(mask, mask, 200, 255, THRESH_BINARY)` keeps only pixels that are clearly foreground. Because shadow pixels carry the value 127, they are discarded here.

3. **Morphology.** Opening and then closing with a 5×5 rectangular kernel removes isolated specks and fills small holes inside objects.

4. **Contours.** `cv::findContours` with `RETR_EXTERNAL` extracts the outer contour of each connected region.

5. **Area filter.** Contours with `contourArea` below `minimumArea` are ignored; for the rest, `cv::boundingRect` gives the `BoundingBox`.

```
cv::Mat foregroundMask;
backgroundSubtractor->apply(image, foregroundMask);
// Remove noise
cv::threshold(foregroundMask, foregroundMask, 200, 255, cv::
    THRESH_BINARY);
cv::Mat kernel = cv::getStructuringElement(cv::MORPH_RECT, cv::Size
    (5, 5));
cv::morphologyEx(foregroundMask, foregroundMask, cv::MORPH_OPEN,
    kernel);
cv::morphologyEx(foregroundMask, foregroundMask, cv::MORPH_CLOSE,
    kernel);
std::vector<std::vector<cv::Point>> contours;
cv::findContours(foregroundMask, contours,
                 cv::RETR_EXTERNAL, cv::CHAIN_APPROX_SIMPLE);
for (const auto& contour : contours)
{
    double area = cv::contourArea(contour);
    if (area < minimumArea)
        continue;
    cv::Rect rect = cv::boundingRect(contour);
    BoundingBox box(rect.x, rect.y, rect.width, rect.height);
    detections.push_back(box);
}
```

Listing 3.9: `MotionDetector::detect()` (core of the method).

3.6.2 Object Tracking

The tracker uses the centroid of each box,

$$\mathbf{c} = \left(x + \frac{w}{2},\; y + \frac{h}{2} \right), \tag{3.1}$$

and the Euclidean distance between an object's last centroid $\mathbf{c}_o$ and a detection's centroid $\mathbf{c}_d$,

$$d(\mathbf{c}_o, \mathbf{c}_d) = \sqrt{(c_{o,x} - c_{d,x})^2 + (c_{o,y} - c_{d,y})^2}. \tag{3.2}$$

In every frame, each existing object (taken in ascending ID order) selects the closest detection that has not been used yet. If $d < d_{\max}$ (80 pixels by default), the object is updated with that detection; otherwise its missed-frame counter is incremented. Detections left unmatched start new objects with the next free ID. Finally, objects whose counter exceeds the limit (5 frames by default) are erased from the map. This is a simple *greedy nearest-neighbour* strategy; the matching loop is shown in Listing 3.10.

```
for (auto& pair : objects)
{
    TrackedObject& object = pair.second;
    double closestDistance = std::numeric_limits<double>::max();
    int closestIndex = -1;
    for (int i = 0; i < detections.size(); i++) {
        if (detectionUsed[i])
            continue;
        cv::Point2f objectCenter = object.getCurrentCentroid();
        cv::Point2f detectionCenter = detections[i].getCentroid();
        double dx = objectCenter.x - detectionCenter.x;
        double dy = objectCenter.y - detectionCenter.y;
        double distance = std::sqrt(dx * dx + dy * dy);
        if (distance < closestDistance) {
            closestDistance = distance;
            closestIndex = i;
        }
    }
    if (closestIndex != -1 && closestDistance < maxMatchDistance) {
        object.update(detections[closestIndex], timestamp);
        detectionUsed[closestIndex] = true;
    }
    else {
        object.incrementMissedFrames();
    }
}
```

Listing 3.10: Matching loop of `ObjectTracker::update()`.

3.6.3 Motion Analysis

From the previous and current centroids, the analyser computes the displacement $(\Delta x, \Delta y)$, the distance moved $s = \sqrt{\Delta x^2 + \Delta y^2}$ and the speed

$$v = \frac{s}{\Delta t} \quad \text{[pixels/second]}, \tag{3.3}$$

where Δt is the difference between the two video timestamps stored in the object (`getDeltaTime()`). Using real timestamps, instead of assuming a fixed frame rate, keeps the speed meaningful if the frame rate changes. The direction is decided by the dominant axis of motion and a movement threshold θ (2 pixels by default). Image y-coordinates grow downwards, so a negative Δy means `UP`. The status is `MOVING` when $s > \theta$ and $v > 0$, and otherwise `STATIONARY`, in which case the direction is also reset to `STATIONARY` (Listing 3.11).

```
std::string direction = "STATIONARY";
if (std::abs(dx) > std::abs(dy)) {
    if (dx > movementThreshold)       direction = "RIGHT";
    else if (dx < -movementThreshold) direction = "LEFT";
}
else {
    if (dy > movementThreshold)       direction = "DOWN";
    else if (dy < -movementThreshold) direction = "UP";
}
const std::string status = distance > movementThreshold && speed > 0.0
    ? "MOVING" : "STATIONARY";
if (status == "STATIONARY")
    direction = "STATIONARY";
object.setMotion(speed, direction, status);
```

Listing 3.11: Direction and status logic of `MotionAnalyzer::analyze()`.

3.6.4 Visualisation and Display

`Visualizer::drawObject()` draws the green bounding box, the red centroid dot and a white label placed just above the box (clamped so it stays inside the image), and `drawTrajectory()` joins the stored centroids with blue line segments. Before display, `main()` computes a scale factor $\min\{1,\ 1280/\text{width},\ 720/\text{height}\}$ and shrinks large frames with `cv::INTER_AREA`, so that high-resolution videos fit on screen. Each log line printed to the console has the format

```
frame=<n> id=<id> x=<x> y=<y> speed_px_s=<v> direction=<d> status=<s> missed=<m>
```

3.6.5 Configuration Parameters

The tunable parameters of each component and their default values are listed in Table 3.5.

Parameter	Value	Meaning
`minArea` (`MotionDetector`)	500 px^2	Smallest contour area accepted as an object
Foreground threshold	200	Removes shadow (127) and weak foreground pixels
Morphology kernel	5×5	Rectangular kernel for opening and closing
`maxDistance` (`ObjectTracker`)	80 px	Largest centroid distance for a match
`maxMissed` (`ObjectTracker`)	5 frames	Frames an unmatched object survives
`threshold` (`MotionAnalyzer`)	2.0 px	Movement needed per frame to count as moving
Display limit	1280×720	Maximum size of the displayed window

Table 3.5: Tunable parameters and their default values

3.7 Build and Execution

The project is configured with CMake. `CMakeLists.txt` sets C++17, requires MSVC, locates OpenCV 5 through `OpenCV_DIR` (default `C:/opencv/build`) and links the eight source files into one executable. For Visual Studio debugging it also adds the OpenCV `bin` directory to the `PATH` and sets the working directory to the source folder, so that the default `Video.mp4` is found. The commands used are shown in Listing 3.12.

```
cmake --preset msvc-x64
cmake --build --preset msvc-debug

ObjectTrackingEngine.exe                    (plays Video.mp4)
ObjectTrackingEngine.exe path\to\clip.mp4
```

Listing 3.12: Building and running the project (Developer PowerShell).

If the video cannot be opened, the program prints an error message and exits with code -1. While running, pressing `q` in the video window ends the program.

3.8 Results and Screenshots

The program was tested on a traffic-camera clip showing a multi-lane road with a bus stop. Figures 3.7–3.10 show frames from the live window. The on-screen clock in the video (bottom right) gives the position in the clip.

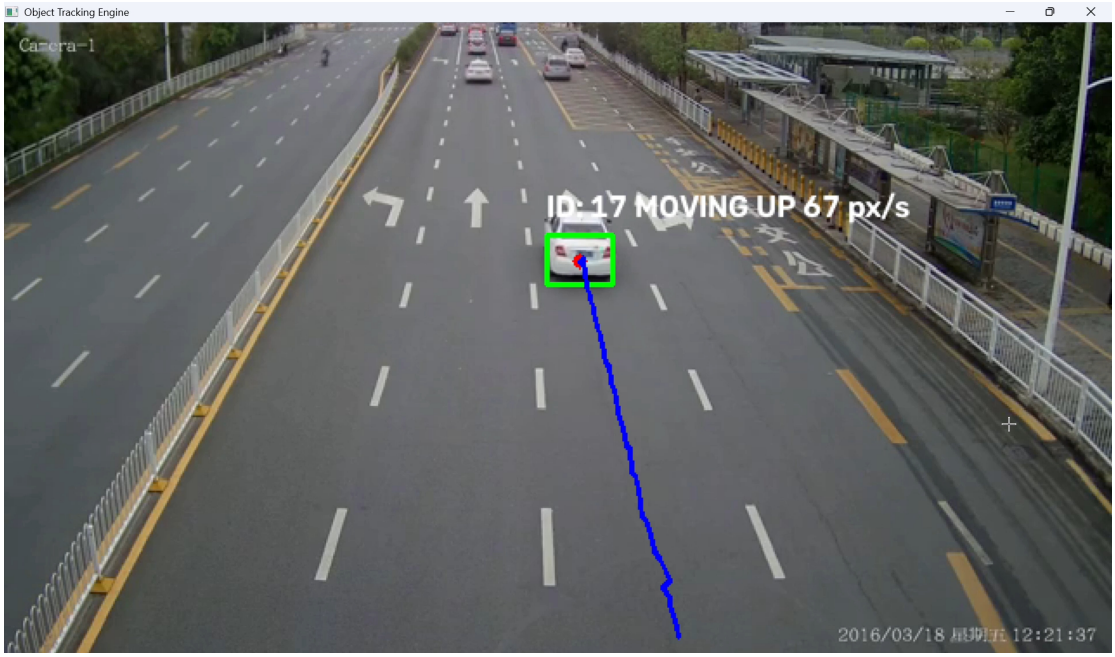


Figure 3.7: A single vehicle (ID 17, clip time 12:21:37) with its bounding box, centroid, label and a long blue trajectory. It is reported as `MOVING UP` at 67 px/s.

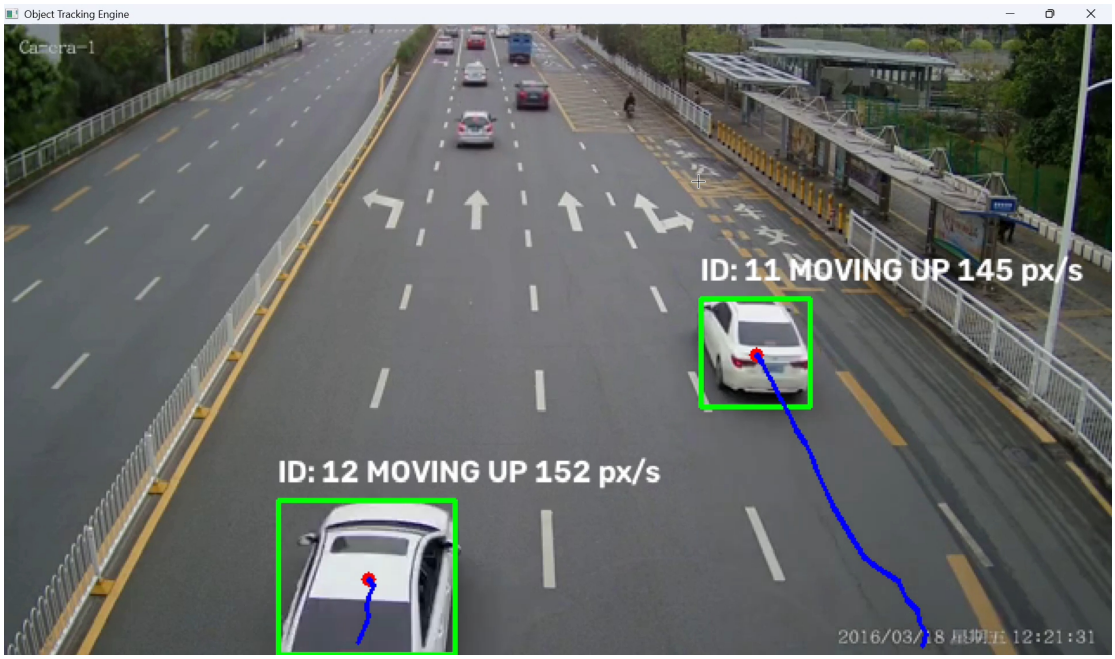


Figure 3.8: Two simultaneous objects (IDs 11 and 12, clip time 12:21:31) with independent IDs, labels and trajectories, both `MOVING UP`. The nearer car (ID 12) shows a larger pixel speed (152 px/s) than the farther one (ID 11, 145 px/s).

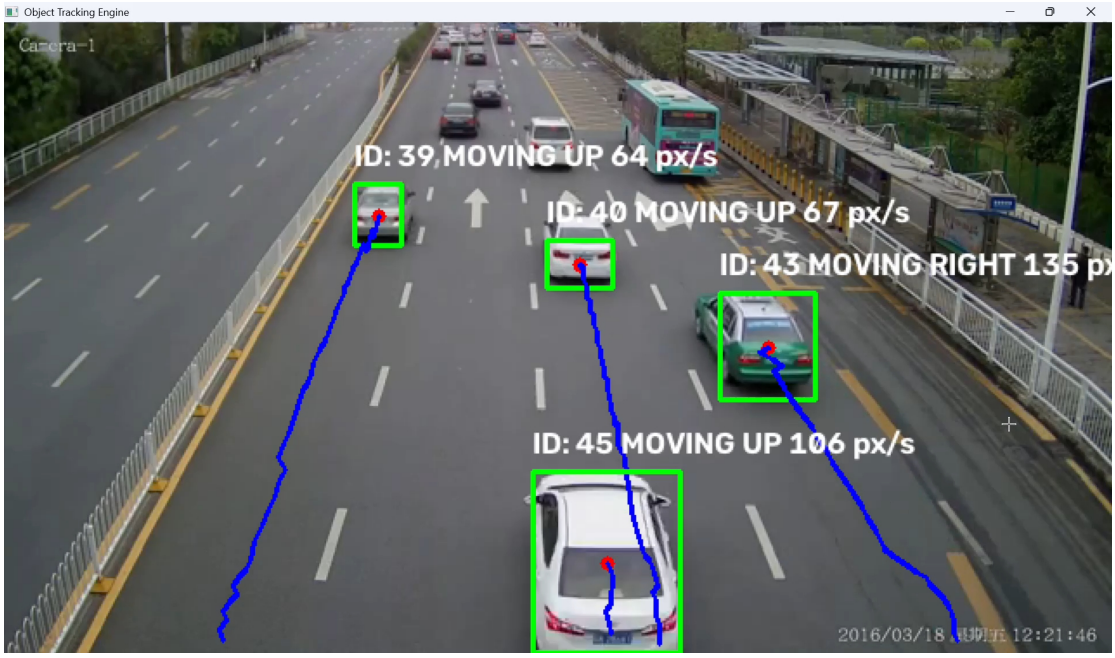


Figure 3.9: Four vehicles tracked at once (IDs 39, 40, 43 and 45, clip time 12:21:46). Vehicle 43, a taxi moving towards the right-hand lane, is labelled `MOVING RIGHT` at 135 px/s, while the others are `MOVING UP`.

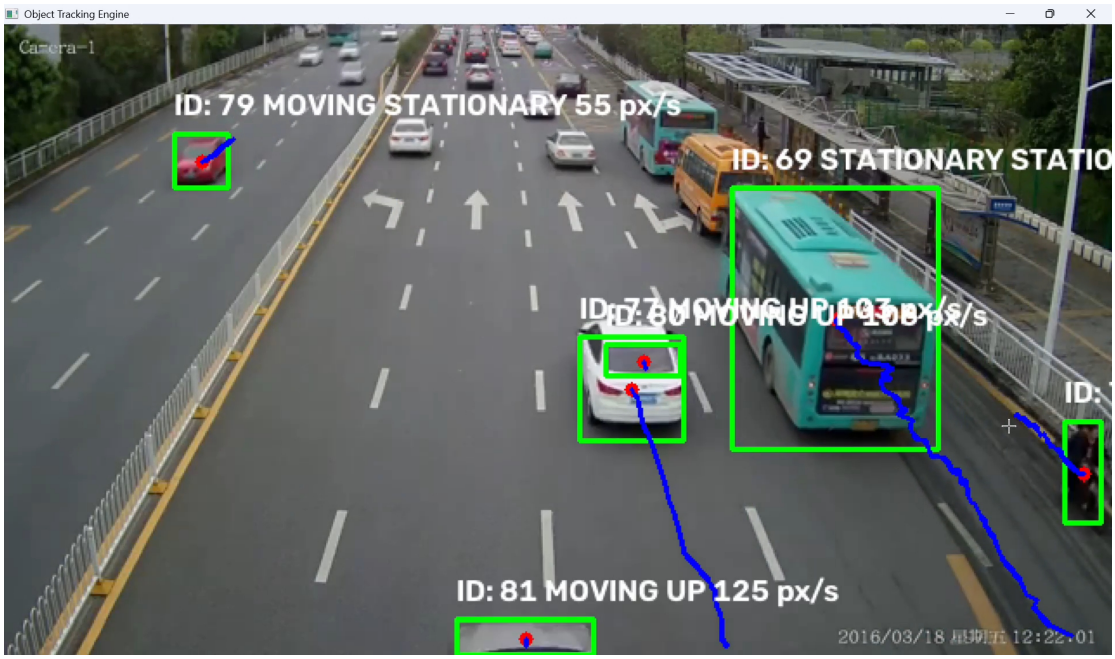


Figure 3.10: A busier frame (clip time 12:22:01). The bus at the stop (ID 69) is labelled `STATIONARY`; other vehicles and a pedestrian at the right edge are tracked at the same time. Two overlapping boxes on the white car (IDs 77 and 80) show a case where one vehicle was detected as two regions.

3.8.1 Observations

- **Correct identities over time.** In the less crowded frames each vehicle keeps one ID and its trajectory follows the road lane, with a smooth path from the bottom of the image (nearer the camera) upwards.
- **Direction and status.** Most vehicles are labelled `MOVING UP`, matching the traffic direction, while the lane-changing taxi in Figure 3.9 is reported as `MOVING RIGHT`, and the bus halted at the stop is reported as `STATIONARY`.
- **Perspective effect.** Speeds are measured in image pixels (for example, the nearer car in Figure 3.8 shows a higher value than the farther one). Because the camera looks down the road, objects near the bottom of the image cover more pixels per second than distant ones moving equally fast, so values are comparable only for the same image region.
- **Challenging cases.** In Figure 3.10, a single car is covered by two boxes (IDs 77 and 80) and the label of ID 79 reads `MOVING STATIONARY`. The latter happens when the distance moved slightly exceeds the 2-pixel threshold while the dominant axis shows less than 2 pixels of motion, so the status becomes `MOVING` but no direction is assigned.

3.9 Limitations and Future Work

3.9.1 Limitations

- **Greedy matching.** Objects choose detections one by one in ID order, without a global optimum. Two close objects, or an occlusion, can cause an ID switch.
- **No motion prediction.** The tracker compares a detection with an object's *last* position; it does not predict where the object should be, so fast objects need a large matching radius.
- **Fragmented detections.** One large or partly occluded object can split into several contours, producing duplicate IDs (Figure 3.10).
- **Background absorption.** MOG2 slowly learns stationary objects into the background, so a vehicle that stops for a long time may eventually stop being detected.
- **Pixel units.** Speed is reported in pixels per second without perspective correction or calibration.
- **Direction labelling.** The rule uses only the dominant axis and a threshold, which can yield the combination `MOVING` / `STATIONARY` for very small motions.
- **Unbounded trajectories.** Each object stores its entire path; this is acceptable for short clips but grows for very long-lived objects.

3.9.2 Future Work

- Replace greedy matching with the **Hungarian algorithm** for optimal assignment of detections to objects.
- Add a **Kalman filter** per object to predict its next position, smooth the trajectory and bridge short occlusions (as in SORT-style trackers).
- Introduce an abstract `Detector` base class with MOG2 and deep-learning (for example YOLO) implementations, which would also bring genuine inheritance and polymorphism into the project's own class hierarchy.
- Merge overlapping boxes and limit the stored trajectory length.
- Calibrate the camera (homography) to report speed in kilometres per hour, add object classification and per-lane counting, and save the annotated video to disk.

3.10 Conclusion

The Object Tracking Engine demonstrates how a non-trivial computer-vision pipeline can be organised into small, cohesive C++ classes. Detection, tracking, analysis and visualisation are separated behind simple interfaces; state is encapsulated; ownership is expressed through composition and standard containers; and the code uses constructors with default arguments, operator overloading, const-correctness and an abstract OpenCV interface. The results on real traffic footage show stable IDs, plausible trajectories and

sensible direction and status labels, and the identified limitations point to clear next steps.

References

1. OpenCV documentation, *Background Subtraction* and *Structural Analysis and Shape Descriptors*. https://docs.opencv.org/

2. Z. Zivkovic, "Improved adaptive Gaussian mixture model for background subtraction," *Proc. ICPR*, 2004.

3. Z. Zivkovic and F. van der Heijden, "Efficient adaptive density estimation per image pixel for the task of background subtraction," *Pattern Recognition Letters*, 27(7), 2006.

4. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, 2(1–2), 1955.

5. R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, 82(1), 1960.

6. A. Bewley et al., "Simple online and realtime tracking," *Proc. IEEE ICIP*, 2016.

7. B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.